Temporal Difference Learning

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW2 is out and is due 10/8!

Check the course website for Office Hours:
https://harvard-cs-1840.github.io/2026/

Recap: Contextual Bandit

(Thanks for the feedback question!)

Finite horizon MDP with $H = 1$ (Contextual Bandit)

$$\mathcal{M} = \{\mathcal{X}, \mathcal{A}, r, H = 1, \mu\}$$

Recap: Contextual Bandit

(Thanks for the feedback question!)

Finite horizon MDP with $H = 1$ (Contextual Bandit)

$$\mathcal{M} = \{\mathcal{X}, \mathcal{A}, r, H = 1, \mu\}$$

Finite horizon MDP with $H > 1$ (Normal MDP)

$$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, r, H, P, \mu_0\},$$
$$r : S \times A \mapsto [0,1], H \in \mathbb{N}^+, P : S \times A \mapsto \Delta(S), s_0 \sim \mu_0$$

Recap: Bellman equation (consistency)

Bellman Eq

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^{\pi}(s')\right], \forall s$$

An iterative approach for estimating V^{π}

$$V^{t+1} \Leftarrow R + \gamma P V^{t}$$

Recap: Bellman equation (consistency)

Bellman Eq

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^{\pi}(s')\right], \forall s$$

An iterative approach for estimating V^{π}

$$V^{t+1} \Leftarrow R + \gamma P V^{t}$$

1. Need to know the transition

Recap: Bellman equation (consistency)

Bellman Eq

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^{\pi}(s')\right], \forall s$$

An iterative approach for estimating V^{π}

$$V^{t+1} \Leftarrow R + \gamma P V^{t}$$

1. Need to know the transition
2. Only works for discrete small MDPs

Today

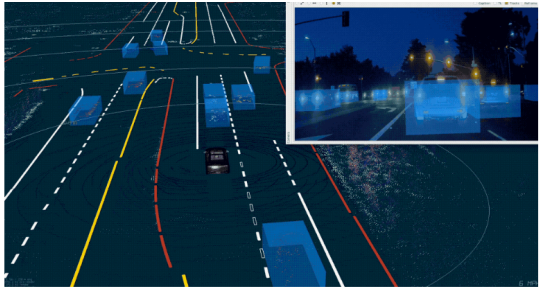
Given MDP $\mathscr{M} = (S, A, r, P, \gamma)$ & a $\pi : S \mapsto \Delta(A)$,

how to estimate $V^\pi(s), \forall s$ WITHOUT knowing P
(i.e., how to learn V^π from experience)

Motivation



our opponent's strategy is unknown



hard to model the transitions of the other drivers/ pedestrians/cyclists)



Hard to model users' reactions to recommendations

Outline:

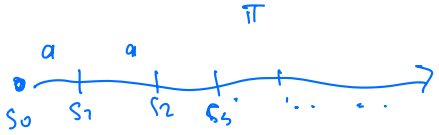
1. Simple Monte Carlo methods

2. Temporal Difference Learning

MC estimation

Setup: we have MDP $\mathscr{M} = (S, A, P, \gamma, r)$, and ***stochastic*** π, i.e., $a \sim \pi(\cdot \mid s)$

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r_h \,\middle|\, \pi, s_0 = s\right]$$



MC estimation

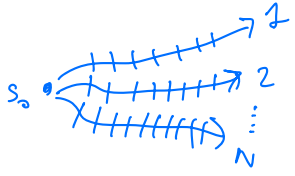
Setup: we have MDP $\mathscr{M} = (S, A, P, \gamma, r)$, and ***stochastic*** π, i.e., $a \sim \pi(\cdot \mid s)$

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r_h \,\middle|\, \pi, s_0 = s\right]$$

MC methods replace Expectation by Sample Average

MC estimation

For all state $s \in \mathcal{S}$:



Generate N i.i.d trajectories from $s_0 = s$ using π

MC estimation

For all state $s \in \mathcal{S}$:

Generate N i.i.d trajectories from $s_0 = s$ using π

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^h r_{i,h} \right]$$

MC estimation

For all state $s \in \mathcal{S}$:

Generate N i.i.d trajectories from $s_0 = s$ using π

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^h r_{ih} \right]$$

Reward at time h at the i-th trajectory

MC estimation

For all state $s \in \mathcal{S}$:

Generate N i.i.d trajectories from $s_0 = s$ using π

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^h r_{i,h} \right]$$

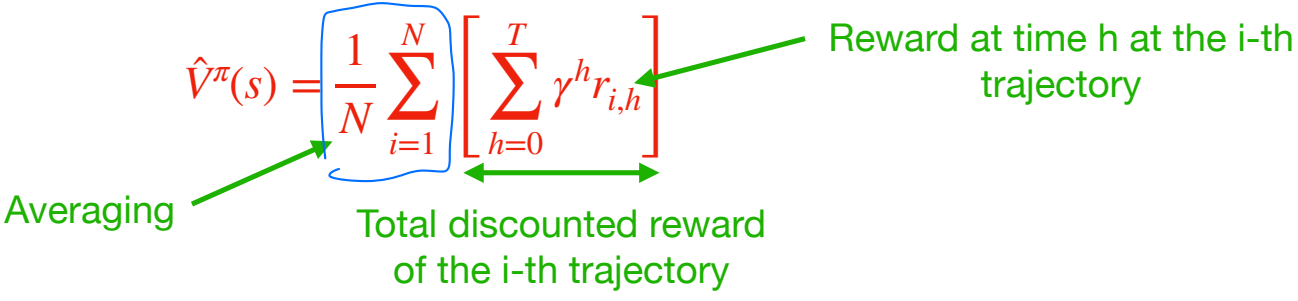
Reward at time h at the i-th trajectory

Total discounted reward of the i-th trajectory

MC estimation

For all state $s \in \mathcal{S}$:

Generate N i.i.d trajectories from $s_0 = s$ using π



MC estimation

For all state $s \in \mathcal{S}$:

T needs to be large enough (i.e., $\gamma^T \approx 0$)

Generate N i.i.d trajectories from $s_0 = s$ using π

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^h r_{i,h} \right]$$

Reward at time h at the i-th trajectory

Averaging

Total discounted reward of the i-th trajectory

$$\sum_{h=0}^{T} \gamma^h r_h + \sum_{h=T+1}^{\infty} \gamma^h r_h$$

MC estimation

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^h r_{i,h} \right]$$

When $T \to \infty$ and $N \to \infty$, we have $\hat{V}^{\pi}(s) \to V^{\pi}(s)$

$G_i^T = \sum_{h=0}^{T} \gamma^h r_{ih}$

$\lim_{T \to \infty} G_i^T \to G_i := \sum_{h=0}^{\infty} \gamma^h r_{ih}$

$\lim_{N \to \infty} \frac{1}{N} \sum_{i=1}^{N} G_i^T \to \mathbb{E}_{\pi}[G^T | s_0 = s]$

MC estimation

$$\hat{V}^{\pi}(s) = \frac{1}{N}\sum_{i=1}^{N}\left[\sum_{h=0}^{T}\gamma^{h} r_{i,h}\right]$$

When $T \to \infty$ and $N \to \infty$, we have $\hat{V}^{\pi}(s) \to V^{\pi}(s)$

The future after T-step has near zero contribution

γ^h

Assume $\gamma = 0.5$

Full return $G = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \ldots = 2$

$G^3 = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = 1.8$

Full return $G - G^3 = .2$

MC estimation

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^{h} r_{i,h} \right]$$

When $T \to \infty$ and $N \to \infty$, we have $\hat{V}^{\pi}(s) \to V^{\pi}(s)$

The future after T-step has near zero contribution

Law of large number

MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

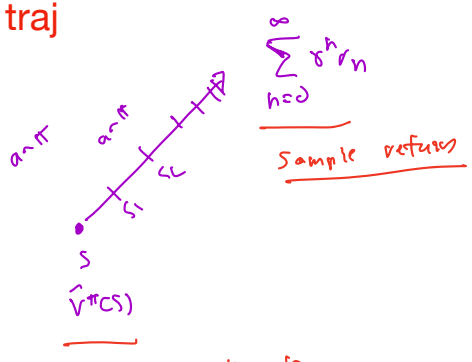
Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

learning rate

target

prediction



MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

total reward of one traj

MC estimation with incremental update

Instead of waiting for all N trajs, we can also update the estimator incrementally every traj

Initialize $\hat{V}^{\pi}(s) = 0, \forall s$

For all state $s \in \mathcal{S}$:

Generate one traj from $s_0 = s$ using π;

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

η: learning rate

total reward of one traj

MC estimation with incremental update

The incremental update is performing
Stochastic gradient descent (SGD)

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

$\min_x \ell(x)$

GD: $x - \eta \cdot \nabla \ell(x)$

SGD: $x - \eta \cdot \widehat{\nabla} \ell(x)$

$\mathbb{E}[\widehat{\nabla} \ell(x)] = \nabla \ell(x)$

MC estimation with incremental update

The incremental update is performing **Stochastic gradient descent (SGD)**

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

$$\ell(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - V^{\pi}(s) \right)^2$$

estimate

ground truth

MC estimation with incremental update

The incremental update is performing **Stochastic gradient descent (SGD)**

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^{\pi}(s) \right]$$

$$\ell(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - V^{\pi}(s) \right)^2$$

$$\nabla \ell(x)\,|_{x=\hat{V}^{\pi}(s)} = 2 \left(\hat{V}^{\pi}(s) - V^{\pi}(s) \right)$$

↑ prediction ↓ target

MC estimation with incremental update

The incremental update is performing **Stochastic gradient descent (SGD)**

$$\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left[\left(\sum_{h=0}^{\infty} \gamma^h r_h \right) - \hat{V}^\pi(s) \right]$$

$$\widetilde{\nabla} \ell(x) \big|_{x=\hat{V}^\pi(s)} = 2 \left(\hat{V}^\pi(s) - \sum_{h=0}^{\infty} \gamma^h r_h \right)$$

$$\hat{V}^\pi(s) - \eta \, \widetilde{\nabla} \ell(x) \big|_{x=\hat{V}^\pi(s)}$$

$$\ell(\hat{V}^\pi(s)) := \left(\hat{V}^\pi(s) - V^\pi(s) \right)^2$$

$$\nabla \ell(x) \big|_{x=\hat{V}^\pi(s)} = 2 \left(\hat{V}^\pi(s) - V^\pi(s) \right)$$

To get a stochastic gradient, replace $V^\pi(s)$ by its unbiased estimate $\sum_{h=0}^{\infty} \gamma^h r_h$

$$\mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r_h \right] = V^\pi(s)$$

Summary

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^{h} r_{i,h} \right]$$

MC estimation is simple, and is based on well-established statistical guarantee

Summary

$$\hat{V}^{\pi}(s) = \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{h=0}^{T} \gamma^{h} r_{i,h} \right]$$

MC estimation is simple, and is based on well-established statistical guarantee

However

It has **high variance**, i.e., total reward on an extremely long trajectory can have a lot of randomness;

Need to **wait** for completing full trajectories to update/estimate $V^{\pi}(s)$

Outline:

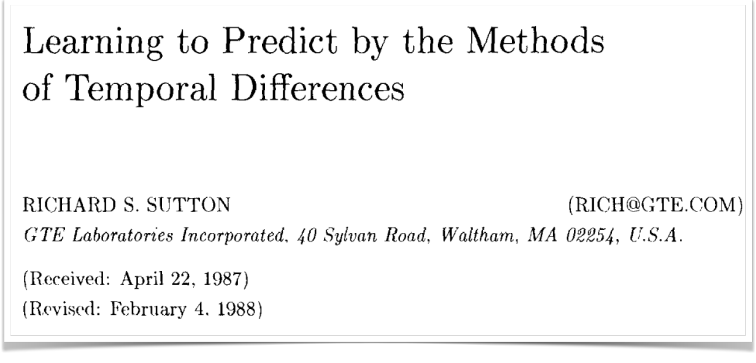


1. Simple Monte Carlo methods

2. Temporal Difference Learning

TD Learning

History: developed by Rich Sutton back in 1988

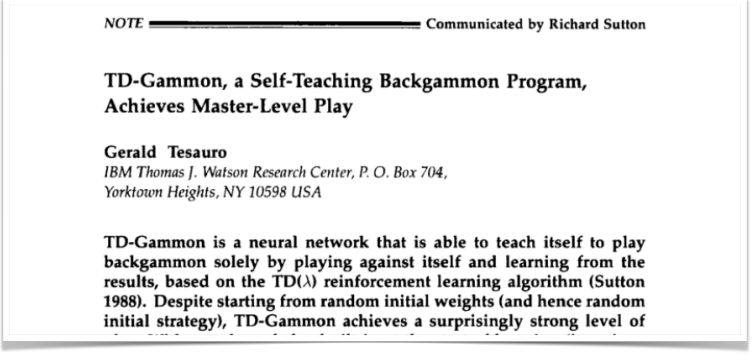


Learning to Predict by the Methods of Temporal Differences

RICHARD S. SUTTON (RICH@GTE.COM)
GTE Laboratories Incorporated, 40 Sylvan Road, Waltham, MA 02254, U.S.A.

(Received: April 22, 1987)
(Revised: February 4, 1988)

History: TD-Gammon in 1994



NOTE Communicated by Richard Sutton

TD-Gammon, a Self-Teaching Backgammon Program, Achieves Master-Level Play

Gerald Tesauro
IBM Thomas J. Watson Research Center, P. O. Box 704, Yorktown Heights, NY 10598 USA

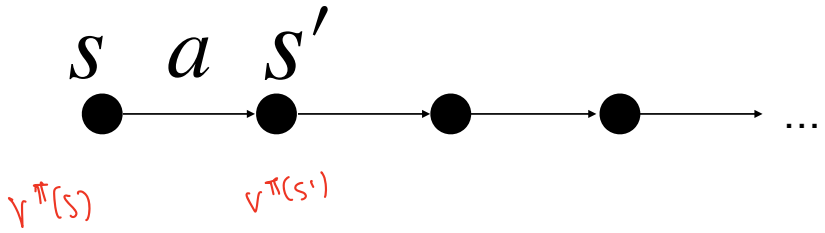
TD-Gammon is a neural network that is able to teach itself to play backgammon solely by playing against itself and learning from the results, based on the TD(λ) reinforcement learning algorithm (Sutton 1988). Despite starting from random initial weights (and hence random initial strategy), TD-Gammon achieves a surprisingly strong level of

TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*

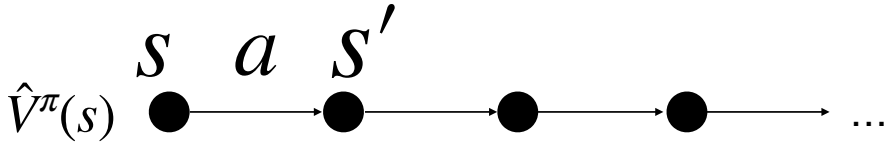
TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



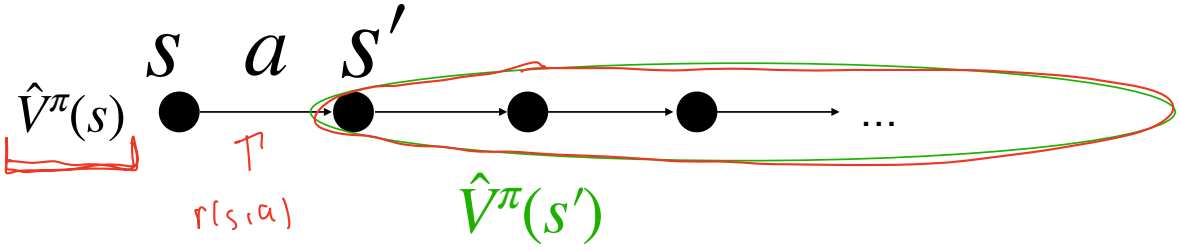
TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



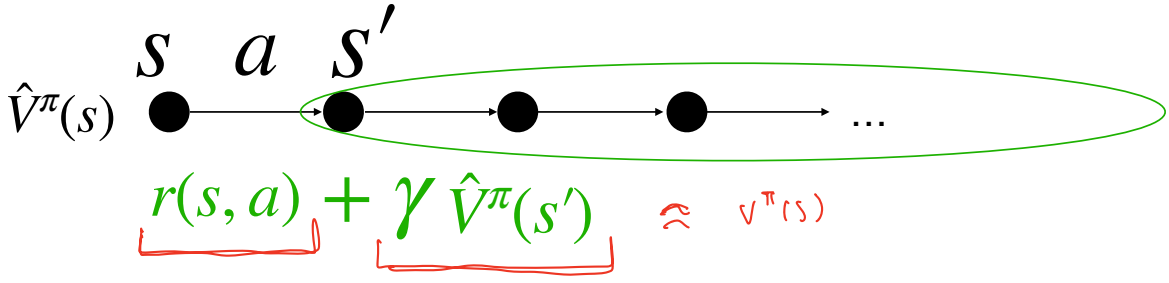
TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



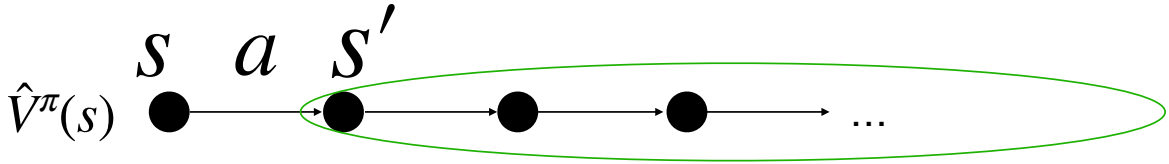
TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



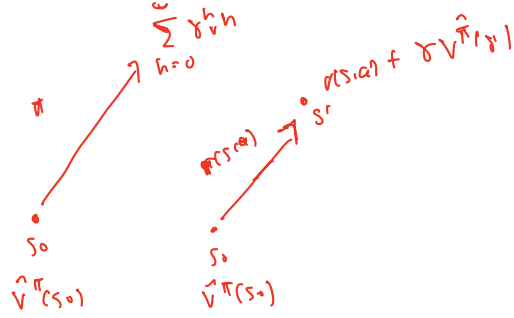
TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



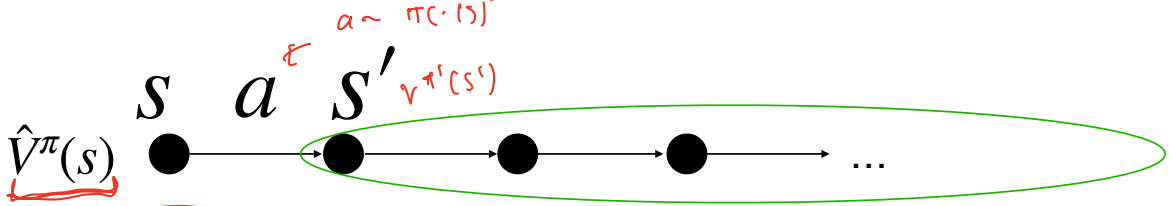
$$r(s,a) + \gamma \hat{V}^{\pi}(s')$$

New target value (bootstrapping)



TD Learning

TD relies on the idea of **Bootstrapping** — *using predictions from our estimator to update the estimator itself*



TD update: $\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r(s,a) + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s) \right)$

TD - error

TD - target

TD Learning

Initialize $V^{\pi}(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$

TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form TD target $r + \gamma \hat{V}^\pi(s')$
>
> Bootstrapping

TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form TD target $r + \gamma \hat{V}^\pi(s')$
>
> Update for s: $\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$
>
> TD-target
>
> prediction
>
> learning rate

TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

> Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$
>
> Form TD target $r + \gamma \hat{V}^\pi(s')$
>
> Update for s: $\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$
>
> Set $s \Leftarrow s'$

TD Learning

Initialize $V^\pi(s) = 0, \forall s$. Set initial state $s \in \mathcal{S}$

While True:

Take action $a \sim \pi(\cdot \mid s)$, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Form TD target $r + \gamma \hat{V}^\pi(s')$

Update for s: $\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$

Set $s \Leftarrow s'$

Remark: (1) we perform online update while interacting w/ MDP;
(2) no need to wait to the end for an update (one update per step)

Interpret TD as “SGD” on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

Interpret TD as “SGD” on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

Interpret TD as “SGD” on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} \hat{V}^{\pi}(s')\right]$$

Interpret TD as “SGD” on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)}\hat{V}^{\pi}(s')\right]$$

This keeps changing as we learning

Interpret TD as "SGD" on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)}\hat{V}^{\pi}(s')\right]$$

This keeps changing as we learning

In-class exercise:

derive one-step SGD update for $\ell_{td}(\hat{V}^{\pi}(s))$

(Hint: how to get an unbiased estimate for y using one transition)

Interpret TD as “SGD” on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^{\pi}(s)) := \left(\hat{V}^{\pi}(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)}\hat{V}^{\pi}(s')\right]$$

This keeps changing as we learning

Interpret TD as "SGD" on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^\pi(s)) := \left(\hat{V}^\pi(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} \hat{V}^\pi(s')\right]$$

This keeps changing as we learning

$$\nabla \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - y\right)$$

$a \sim \pi(\cdot|s)$

$r(s,a)$

$s' \sim P(\cdot|s,a)$

$\hat{V}^\pi(s')$

$r(s,a) + \hat{V}^\pi(s')$

Interpret TD as "SGD" on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^\pi(s)) := \left(\hat{V}^\pi(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} \hat{V}^\pi(s')\right]$$

This keeps changing as we learning

$$\nabla \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - y\right)$$

$$\widetilde{\nabla} \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - \left(r + \gamma \hat{V}^\pi(s')\right)\right)$$

Interpret TD as "SGD" on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^\pi(s)) := \left(\hat{V}^\pi(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} \hat{V}^\pi(s')\right]$$

$$\nabla \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - y\right)$$

$$\widetilde{\nabla} \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - \left(r + \gamma \hat{V}^\pi(s')\right)\right)$$

Unbiased estimate of y

This keeps changing as we learning

Interpret TD as "SGD" on TD loss

TD is not the usual SGD, i.e., it is **not** running SGD to minimize a fixed loss function

TD may be interpreted as running SGD on an **evolving** loss function (TD loss)

$$\ell_{td}(\hat{V}^\pi(s)) := \left(\hat{V}^\pi(s) - y\right)^2, \text{ where } y = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} \hat{V}^\pi(s')\right]$$

$$\nabla \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - y\right)$$

$$\widetilde{\nabla} \ell_{td}(x)|_{x=\hat{V}^\pi(s)} := 2\left(\hat{V}^\pi(s) - \left(r + \gamma \hat{V}^\pi(s')\right)\right)$$

Unbiased estimate of y

This keeps changing as we learning

TD: $\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) - \eta \widetilde{\nabla} \ell_{td}(x)|_{x=\hat{V}^\pi(s)}$

TD Learning

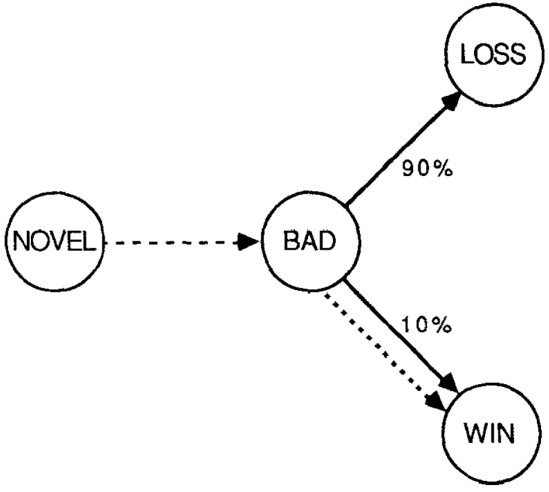
[Informal] Assume π has non-trivial probability of visiting every state.
Setting learning rate η properly, we will have:

$$\hat{V}^{\pi}(s) \rightarrow V^{\pi}(s), \forall s,$$

when # of interactions approaches to ∞

(concrete convergence rates are known as well)

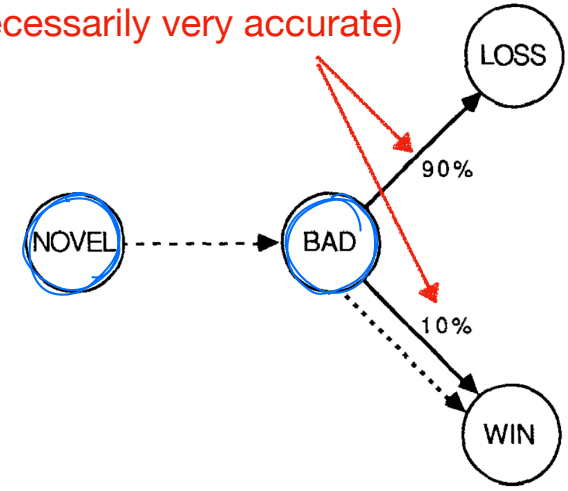
Example of faster learning with TD



[Sutton, 1988]

Example of faster learning with TD

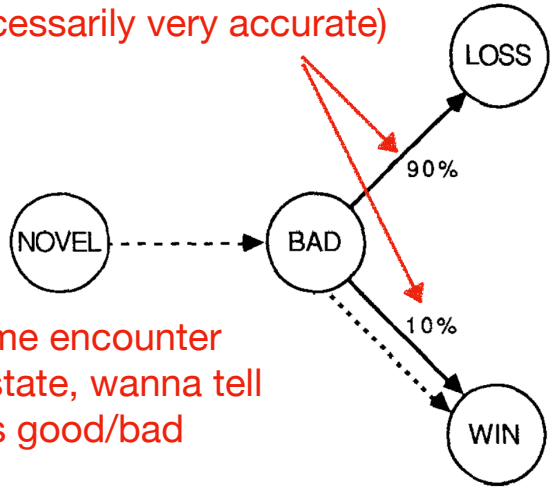
From past game experiences
(not necessarily very accurate)



[Sutton, 1988]

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)

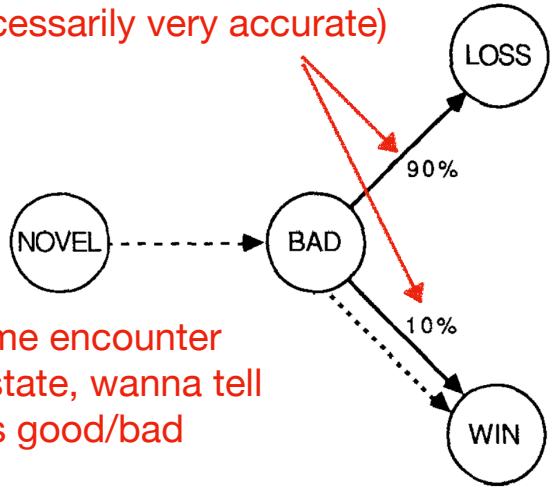


first time encounter
“novel” state, wanna tell
if it’s good/bad

[Sutton, 1988]

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)



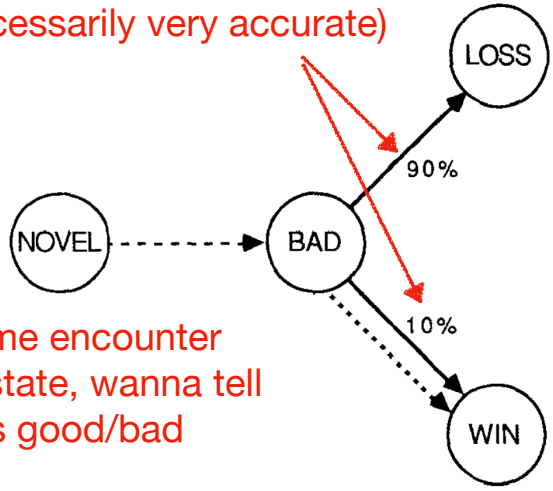
first time encounter
“novel” state, wanna tell
if it’s good/bad

[Sutton, 1988]

MC: have to rollout from NOVEL many times
to get a reasonable estimate of future

Example of faster learning with TD

From past game experiences
(not necessarily very accurate)



first time encounter
“novel” state, wanna tell
if it’s good/bad

[Sutton, 1988]

MC: have to rollout from NOVEL many times to get a reasonable estimate of future

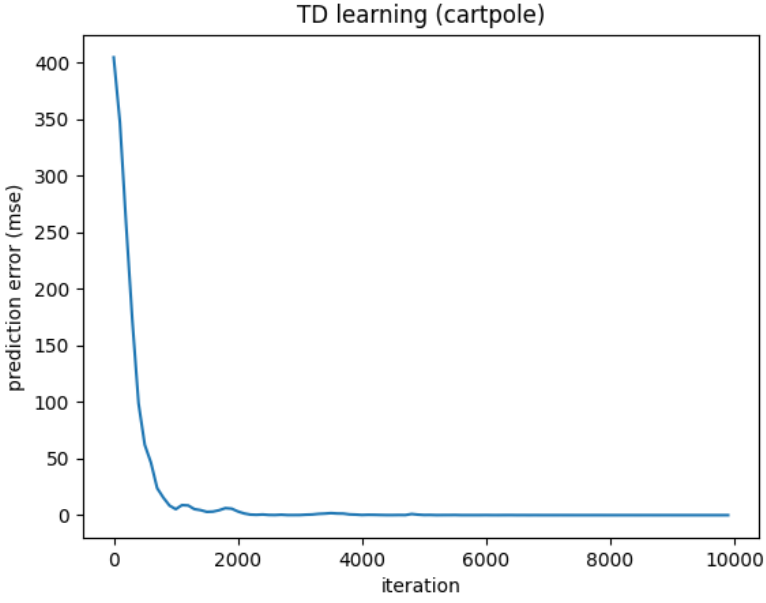
TD (bootstrapping): NOVEL leading to BAD in one-step implies NOVEL is likely bad

TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)

TD learning on CartPole

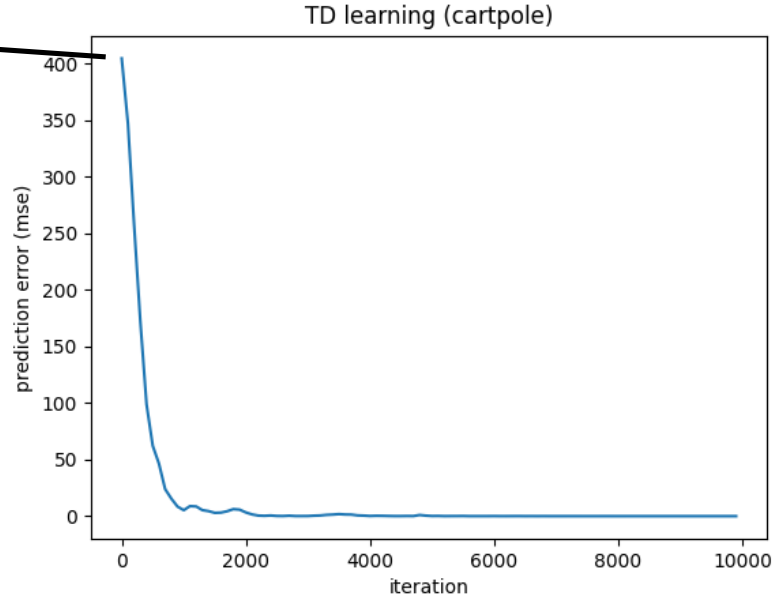
Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)



TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)

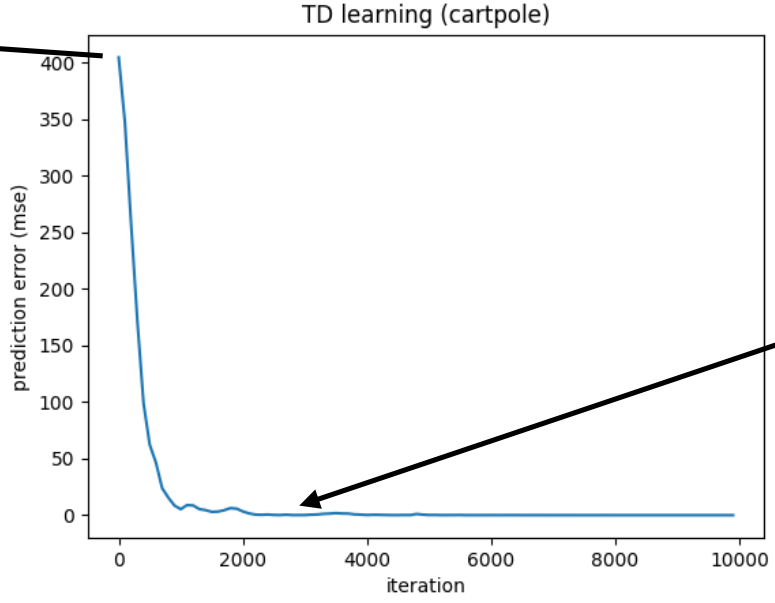
A randomly initialized NN as $\hat{V}^{\pi}(s)$ (very bad estimator)



TD learning on CartPole

Note: Cartpole's state is continuous, so we will need TD w/ function approximation, e.g., neural network (we will get there very soon)

A randomly initialized NN as $\hat{V}^{\pi}(s)$ (very bad estimator)



Almost perfectly estimate the true target $V^{\pi}(s)$

TD vs MC: bias-variance tradeoff

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD vs MC: bias-variance tradeoff

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

TD vs MC: bias-variance tradeoff

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s) \right)$$

TD vs MC: bias-variance tradeoff

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(\underline{r + \gamma \hat{V}^{\pi}(s')} - \hat{V}^{\pi}(s) \right)$$

Randomness in target is
from **one-step transition**

TD vs MC: bias-variance tradeoff

MC is unbiased, but has higher variance (i.e., randomness over an entire traj)

TD can have high bias, but has lower variance

Large bias when $\hat{V}^\pi(s')$ is far from $V^\pi(s')$

$$\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$$

Randomness in target is from **one-step transition**

Summary

TD Learning: an online algorithm for estimating V^{π} on the fly from agent's own experience

Summary

TD Learning: an online algorithm for estimating V^{π} on the fly from agent's own experience

TD Learning uses idea of <u>bootstrapping</u>: using estimator $\hat{V}^{\pi}$'s prediction to improve the estimator $\hat{V}^{\pi}$ itself

Summary

TD Learning: an online algorithm for estimating V^{π} on the fly from agent's own experience

TD Learning uses idea of <u>bootstrapping</u>: using estimator $\hat{V}^{\pi}$'s prediction to improve the estimator $\hat{V}^{\pi}$ itself

Key update step in TD:

Summary

TD Learning: an online algorithm for estimating V^{π} on the fly from agent's own experience

TD Learning uses idea of <u>bootstrapping</u>: using estimator $\hat{V}^{\pi}$'s prediction to improve the estimator $\hat{V}^{\pi}$ itself

Key update step in TD:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta \left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s) \right)$$

Summary

TD Learning: an online algorithm for estimating V^π on the fly from agent's own experience

TD Learning uses idea of <u>bootstrapping</u>: using estimator $\hat{V}^\pi$'s prediction to improve the estimator $\hat{V}^\pi$ itself

Key update step in TD:

$$\hat{V}^\pi(s) \Leftarrow \hat{V}^\pi(s) + \eta \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$$

TD target
(bootstrapping)

Summary

TD Learning: an online algorithm for estimating V^{π} on the fly from agent's own experience

TD Learning uses idea of <u>bootstrapping</u>: using estimator $\hat{V}^{\pi}$'s prediction to improve the estimator $\hat{V}^{\pi}$ itself

Key update step in TD:

$$\hat{V}^{\pi}(s) \Leftarrow \hat{V}^{\pi}(s) + \eta\left(r + \gamma \hat{V}^{\pi}(s') - \hat{V}^{\pi}(s)\right)$$

TD target (bootstrapping)

TD error

Feedback and Attendance

https://bit.ly/4gYn2nC

